

- **CSP Bypass (Content Security Policy Bypass)**
- **DOM-Based Cross-Site Scripting (DOM XSS)**
- **Blind SQL Injection**

The experiments were performed on the **Low, Medium, and High** security levels in DVWA to analyze how different protection mechanisms affect exploitation techniques. The testing environment was implemented using [Kali Linux](#), which provides penetration testing tools commonly used in cybersecurity labs.

The main objective of this assignment is to understand how these vulnerabilities work, how attackers exploit them, and how security controls can reduce or prevent such attacks. Through practical testing, the assignment demonstrates the differences between weak and strong security configurations and highlights the importance of secure coding practices in web application development.

CSP Bypass (Content Security Policy Bypass)

Low Level



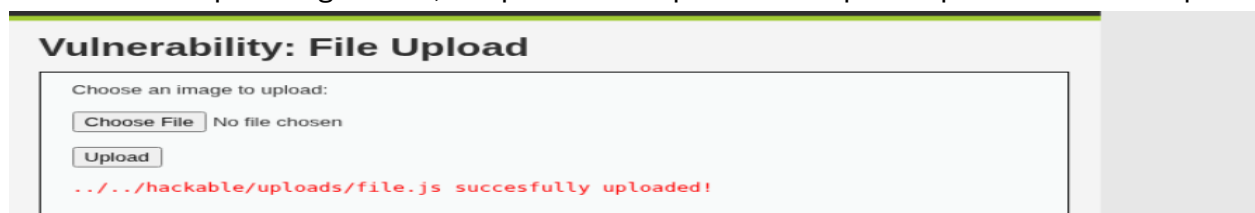
DVWA

Vulnerability: Content Security Policy (CSP) Bypass

You can include scripts from external sources, examine the Content Security Policy and enter a URL to include here:

- **Steps Performed:**

- Opened the CSP page in DVWA.
- First, a direct attempt was made to include a JavaScript file from Pastebin by inserting the URL into the input field. However, the payload did not execute correctly because of a programming issue in the DVWA challenge itself.
- To bypass this restriction, a File Upload vulnerability was used. A malicious JavaScript file named `file.js` was created locally and uploaded to the DVWA server. After uploading the file, the path of the uploaded script was provided as the input.



DVWA

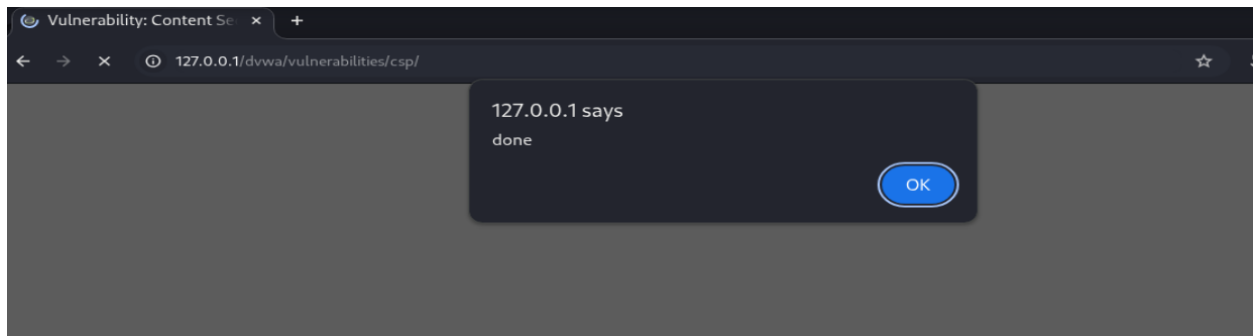
Vulnerability: File Upload

Choose an image to upload:

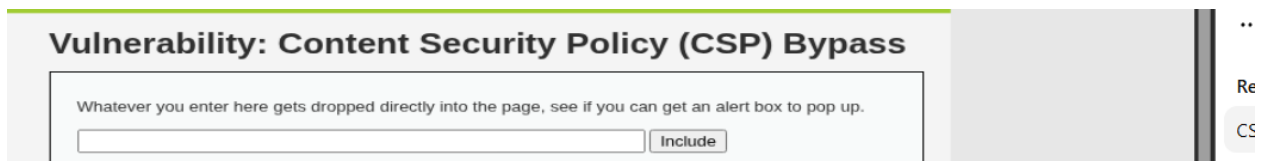
No file chosen

../../hackable/uploads/file.js succesfully uploaded!

After Using `../../hackable/uploads/file.js`



Medium Level



- In the Medium security level, the Content Security Policy used a nonce value inside the `script-src` directive as a protection mechanism. A nonce is a security token that should normally be generated randomly for every request in order to allow only trusted scripts to execute.
 - Content-Security-Policy: `script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXIlgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=';`
- During testing, it was observed that the nonce value remained exactly the same every time the page was refreshed. This meant that the nonce was static and predictable instead of being dynamically generated
- To bypass the CSP protection, the discovered nonce value was copied and added to a malicious script tag
 - [`<script nonce=TmV2ZXIlgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=>alert("done");</script>`]
- By including the correct nonce inside the injected script, the browser treated the unauthorized script as trusted and executed it successfully.

Hard Level

Vulnerability: Content Security Policy (CSP) Bypass

The page makes a call to `../vulnerabilities/csp/source/jsonp.php` to load some code. Modify that page to run your own code.

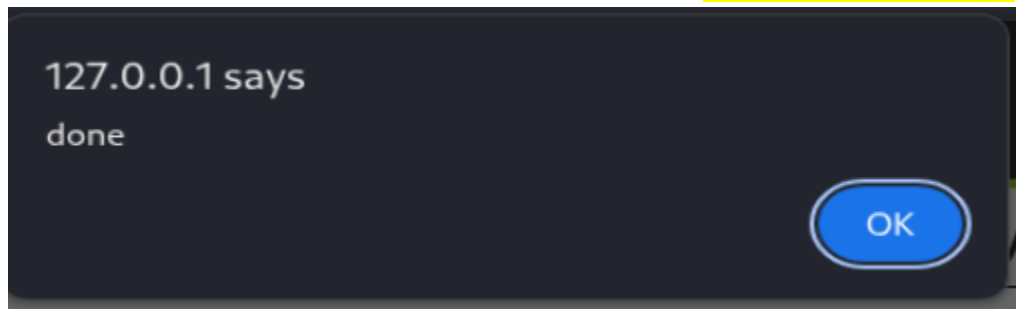
`1+2+3+4+5=15`

Solve the sum

- In the High security level, the CSP bypass was achieved by exploiting a vulnerable callback function implementation in the application.
- The application accepted user-controlled input through a `callback` parameter and inserted this value directly into the server response without proper validation. This created a security weakness because attackers could control the JavaScript function being executed.
- To perform the bypass, the web request was intercepted using a proxy tool such as Burp Suite. The value of the `callback` parameter was then modified to include malicious JavaScript code instead of the expected object or function name.

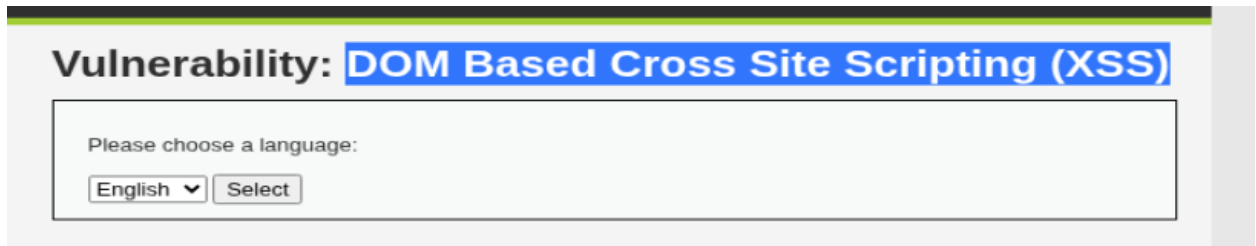
<pre>GET /dwa/vulnerabilities/csp/source/jsonp.php?callback= solveSum HTTP/1.1 Host: 127.0.0.1 sec-ch-ua-platform: "Linux" Accept-Language: en-US,en;q=0.9 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146" User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 sec-ch-ua-mobile: ?0 Accept: */* Sec-Fetch-Site: same-origin</pre>	<pre>1 HTTP/1.1 200 OK 2 Date: Fri, 08 May 2026 01:02:31 GMT 3 Server: Apache/2.4.66 (Debian) 4 Content-Length: 25 5 Keep-Alive: timeout=5, max=100 6 Connection: Keep-Alive 7 Content-Type: application/json; charset=UTF-8 8 9 solveSum({ 10 "answer": "15" 11 })</pre>
---	---

- We modify the call back
- GET /dwa/vulnerabilities/csp/source/jsonp.php?callback=alert("done"); HTTP/1.1



DOM Based Cross Site Scripting (XSS)

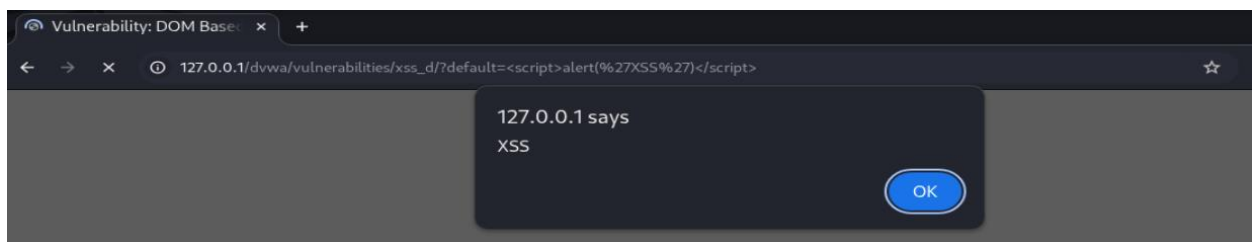
Easy Level



- First inspect and search for english we choose

```
document.write("<option value='English'>English</option>");
document.write("<option value='French'>French</option>");
document.write("<option value='Spanish'>Spanish</option>");
document.write("<option value='German'>German</option>");
<script>
```

- The application takes the value of a URL parameter (like default) and writes it directly into the Document Object Model (DOM) of the page using JavaScript.
- **The Attack:** You can simply append a <script> tag to the URL.
- **Execution:** By changing the URL parameter to ?default=<script>alert('XSS')</script>, the browser executes the script immediately because the page blindly trusts and renders the input as HTML.



Medium Level

Medium Level

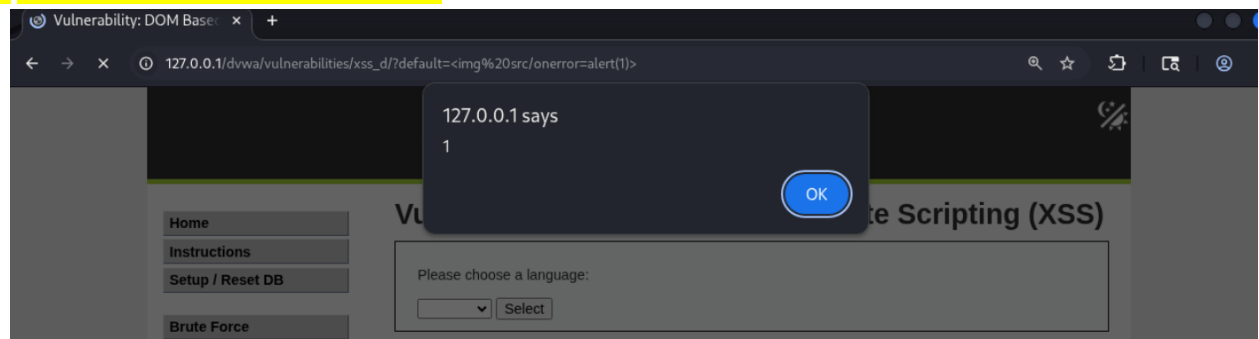
The developer has tried to add a simple pattern matching to remove any references to "<script" to disable any JavaScript. Find a way to run JavaScript without using the script tags.

Spoiler:



In the Medium level, the developer has implemented a **blacklist** filter to block common keywords like <script>.

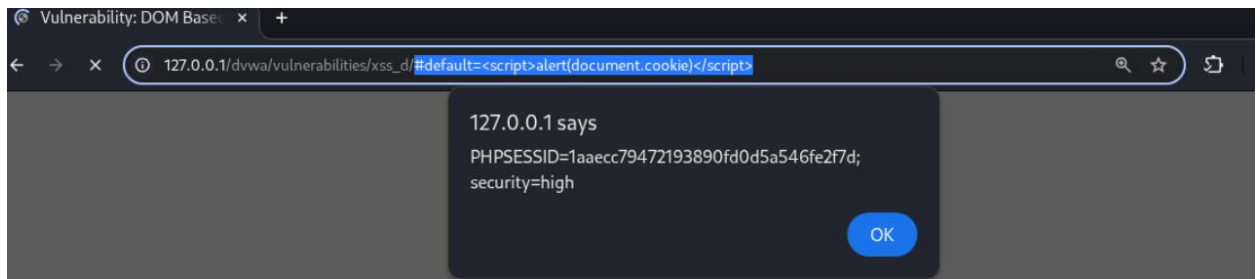
- **The Vulnerability:** Blacklists are often incomplete. While the code looks for specific tags, it might not account for variations or different HTML attributes.
- **The Attack:** Instead of using a script tag, you can use an HTML element with an "event handler."
- **Execution:** we use something from port sweager cheat sheet
- like `<img src/onerror=alert(1)>`



Hard level

The Hard level is more secure because it performs more rigorous checks on the server-side or within the main URL structure to prevent common tags from being processed.

- **The Vulnerability:** The application still processes data after the **Fragment Identifier** (the # symbol in a URL). Data following the # is handled by the browser and is not typically sent to the server.
- **The Attack:** Using the "Fragment" technique allows the payload to bypass server-side security filters entirely.
- **Execution:** The attacker appends the payload after a hashtag, such as `#default=<script>alert(document.cookie)</script>`. Because the server doesn't see the part after the #, it doesn't block the request. However, the client-side JavaScript still reads that part of the URL and injects it into the page, triggering the script.



SQL Injection (Blind)

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

Easy Level

At the Easy level, there is no protection against special characters, allowing for direct injection of Boolean and Time-based payloads.

- **Boolean-Based Logic**

- The goal is to ask the database "True/False" questions. For example, to find out the first letter of a username, you use a query like: `1' AND SUBSTRING((SELECT user FROM users WHERE user_id=1),1,1)='a`.
- If the page displays "User ID exists," the first letter is 'a'. If not, you try 'b', 'c', etc.

- **Time-Based Logic**

- If the page response doesn't change regardless of the query, you use a time delay.
- **Payload:** `1' AND (SELECT 1 FROM (SELECT(SLEEP(5))))a--`
- **Execution:** If the server takes exactly 5 seconds to respond, you know your injected command was executed. This confirms the vulnerability even when the UI is completely silent.

The screenshot shows a web application interface for a SQL Injection challenge. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, and Command Injection. The main content area is titled 'Vulnerability: SQL Injection (Blind)'. It contains a form with a dropdown menu for 'User ID' (currently showing '1 and sleep(5)#') and a 'Submit' button. Below the form, a red message states 'User ID is MISSING from the database.' Below the form is a 'More Information' section. At the bottom of the image, the browser's developer tools are open to the 'Elements' tab, showing the HTML structure of the form. The HTML includes a form with a select element for 'id' with five options: '1 and sleep(5)#', '2', '3', '4', and '5'.

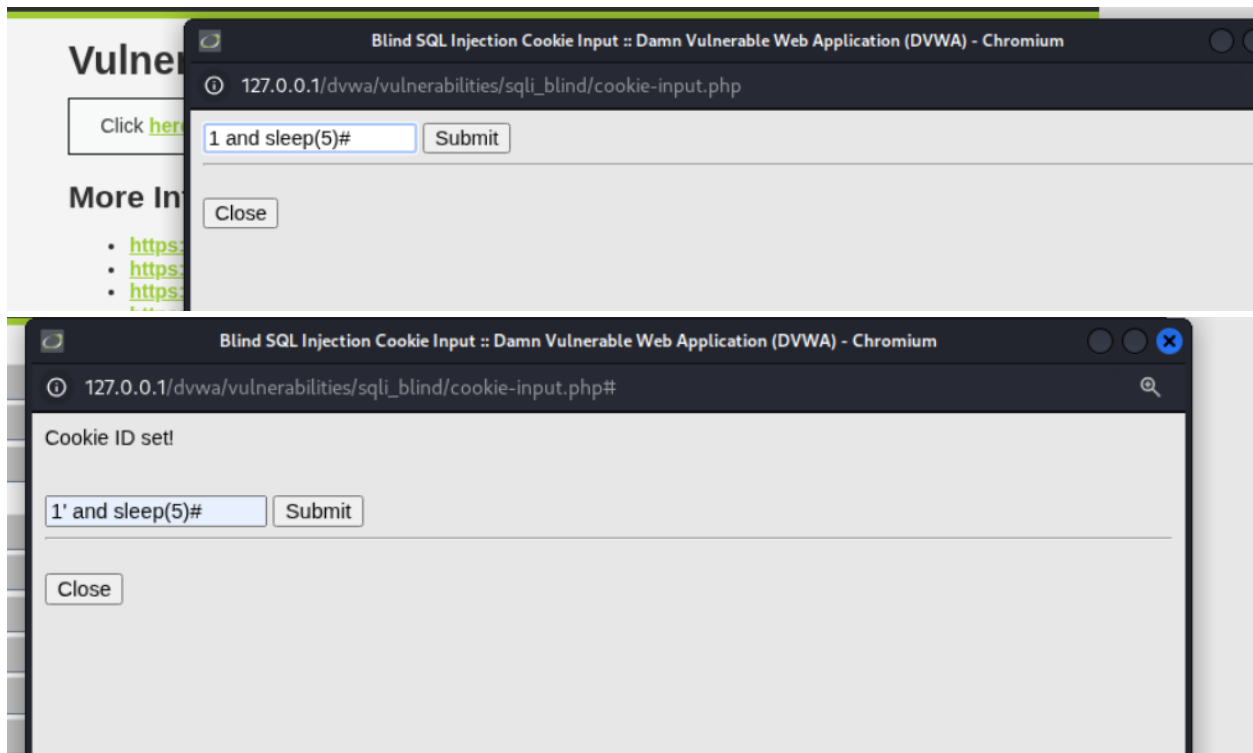
```
<h1>Vulnerability: SQL Injection (Blind)</h1>
<div class="vulnerable_code_area">
  <form action="#" method="POST">
    <p>
      " User ID: "
      <select name="id">
        <option value="1 and sleep(5)#">1 and sleep(5)#</option> == $0
        <option value="2">2</option>
        <option value="3">3</option>
        <option value="4">4</option>
        <option value="5">5</option>
```

Medium Level

The Medium level adds a basic layer of security, typically by using `mysqli_real_escape_string()` to escape quotes ('), which breaks the simple payloads used in the Easy level.

- **The Workaround**

- Since quotes are blocked, the attacker avoids them by using numeric values or different input fields (like a drop-down menu where the value is passed via a POST request).



Hard level

Core Strategy

The Hard level uses a separate page for input to break automated tools. You must use the **hash symbol (#)** to comment out the rest of the developer's query and force the database to listen to your logic.

1. Boolean-Based (Testing Logic)

You verify the vulnerability by checking if the page response changes based on a True/False statement:

- **True:** `1' AND 1=1#` → (Result: "User ID exists")
- **False:** `1' AND 1=2#` → (Result: "User ID is MISSING")

2. Manual Data Extraction (Character by Character)

Since no data is displayed, you must guess the letters of the database or table names one by one:

- **Query:** `1' AND (SELECT SUBSTRING(database(),1,1))='d' #`
- If the result is **True**, the first letter of the database is **'d'**.
- To check the second letter, change it to `SUBSTRING(database(),2,1))='v' #`.

3. Time-Based (The "Sleep" Test)

If the page doesn't give a clear "Exists/Missing" message, use time delays:

- **Query:** `1' AND IF(DATABASE() LIKE 'd%', SLEEP(5), 1) #`
- If the page takes **5 seconds** to load, your guess ('d') is correct.